

CSINTSY MCO2: PINOYBOT

Group Members:

Catapang, Nathan James B.

Hernaez, Raeka Estrella R.

Serrano, Paul Rhazzel

Tamayo, Chelsea Nichole C.

DECLARATION OF AI USAGE

The group used AI tools to better understand the project and to have an overview of the requirements needed to be implemented in the development of PinoyBot. AI was also used to explain concepts and provide insights that helped the members improve the overall performance of the bot. The group carefully reviewed all suggestions before applying them to ensure that they were correct and appropriate based on the project specifications.

AI Tool(s) used:

ChatGPT & Google Gemini

- Asked to review the file and list down the requirements needed for the project.
 - AI provided a clear overview of the requirements and deliverables. This made the specifications easier to comprehend. This also allowed the group to plan the project more effectively and begin the implementation with a clearer understanding of the expected outcomes.
- Asked to suggest suitable classifier models that could be implemented for the project.
 - AI provided recommendations on which was the most ideal classifier model to use for the project. It explained the strengths and limitations of different classifier models and compared their expected performance. This helped the group evaluate the available options before selecting the final classifier model.

Attached below are the screenshots of the chat logs:

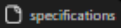
 specifications.pdf
PDF

can you review this file and list down the requirements needed for the project

I reviewed the project specification. Here are the complete requirements for the **PinoyBot (Major Course Output 2)** project.

1. Project Goal

Develop an intelligent **word-level language identification system** for Filipino code-switched text using **supervised machine learning**. The system should identify the language of every word in a sentence.



2. Dataset Requirements

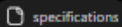
Manual Annotation

Before training the model, your group must manually annotate the dataset using these labels:

Label	Meaning
FIL	Filipino/Tagalog words
ENG	English words
CS	Intra-code switched words (e.g. <i>naglunch</i> , <i>pina-explain</i>)
OTH	Names, abbreviations, numbers, punctuation, emojis, hashtags, etc.

Important annotation rules:

- Filipino loanwords written in Filipino spelling → **FIL**
- English spelling → **ENG**
- Mixed Filipino + English in one word → **CS**
- Names, abbreviations, punctuation, numbers, emojis → **OTH**
- Misspellings should be labeled according to their intended meaning.
- Abbreviations are always **OTH**.





Project Checklist

- ☒ Manually annotate the dataset (`FIL` , `ENG` , `CS` , `OTH`)
- ☒ Extract features for every word
- ☒ Train a supervised ML classifier (e.g., Decision Tree or Naïve Bayes)
- ☒ Use a 70/15/15 train-validation-test split
- ☒ Evaluate using Accuracy, Precision, Recall, and F1-score
- ☒ Save the trained model
- ☒ Complete `tag_language()` in `pinoybot.py`
- ☒ Ensure output format matches the required labels and input length
- ☒ Write a report (maximum 4 pages)
- ☒ Submit the ZIP file (code + trained model + training scripts) and the PDF report.

     ...  Sources

what classifier would be most ideal? and why?

Based on **your project specifications**, **your dataset**, and the nature of **word-level Filipino-English language identification**, here is how I would rank the classifiers.

Rank	Classifier	Recommendation
🏆 1	Random Forest	Best overall choice
🏆 2	Logistic Regression	Very good baseline
🏆 3	Decision Tree	Good but tends to overfit
4	Naïve Bayes	Fast but weaker
5	SVM	High accuracy but slower and harder to tune

Why I recommend Random Forest

Your project is **not** trying to understand sentence meaning.

Instead, it classifies each **individual token** based on engineered features like

- prefixes
- suffixes
- capitalization
- punctuation
- digits
- word length
- vowel/consonant patterns
- hyphen usage

how can we improve on it



There are **three areas** where you can improve your model:

1. **Better features** (biggest improvement)
2. **Better classifier**
3. **Better training and tuning**

For your project, **feature engineering will likely have a larger impact than changing classifiers**, because the specifications emphasize extracting useful features from words.

A. Vowel ratio

Filipino words often have a higher vowel frequency than English.

Example

kumakain



vs

strength



Feature:

Python



Run

```
num_vowels  
vowel_ratio
```

2. Use sentence context

This is probably the **largest improvement** you can make while still following the assignment.

Your dataset contains

sentence_id



which means you know the neighboring words.

The specifications even suggest using properties of surrounding words as features.

6. Try multiple classifiers

Your report becomes much stronger if you compare models instead of using just one.

Example

Model	Accuracy	Macro F1
Decision Tree	90.8%	0.89
Naïve Bayes	91.5%	0.90
Logistic Regression	94.3%	0.94
Random Forest	96.1%	0.96

Then justify why you selected the final model.

i mean more accurate, not more faster

Got it! Accuracy is definitely the primary goal for a machine learning classification task like this.

To make your model more accurate, we need to focus on upgrading the algorithms and handling the distribution of your dataset. Here are the three most effective changes you can make to `training_model.py` to boost your macro F1-score.

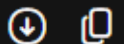
1. Upgrade to a Random Forest Classifier

A single `DecisionTreeClassifier` is prone to overfitting (memorizing the training data instead of learning patterns). An ensemble model like a **Random Forest** builds dozens or hundreds of decision trees and averages their predictions. This almost always results in a massive accuracy boost.

How to implement it:

At the top of `training_model.py`, add this import:

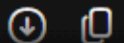
Python



```
from sklearn.ensemble import RandomForestClassifier
```

Then, replace your Decision Tree setup with this:

Python



+ Ask Gemini

Pro ▾



Gemini is AI and can make mistakes.

Create a prompt to give to an LLM (ChatGPT, Google Gemini, Claude) that understands a given python specs file, to create an easy to understand task list / check list that I can go through and check, and whenever I send a block of code it would assist me on what I did wrong and what I can do better, basically a programming assistant that strictly complies with the specs

Copy-paste prompt:

You are my programming assistant for a Python programming project. Your role is to help me strictly follow the specification. 

I will first provide you with the complete Python project specification. Read and analyze it thoroughly. I will then provide you with code blocks. For each code block, you will provide me with a list of tasks to complete and a checklist to verify that the code complies with the specification.

Your tasks are:

1. SPECIFICATION ANALYSIS

- Carefully understand every requirement, restriction, expected output, input format, file structure, and dependencies.
- Identify hidden requirements or details that are easy to miss.
- Do not assume requirements that are not stated unless you clearly label them as assumptions.
- Treat the specification as the highest authority. If my code conflicts with the specification, prioritize the specification.

2. CREATE AN IMPLEMENTATION CHECKLIST

After analyzing the specification, create a clear and easy-to-follow task checklist that breaks down the implementation into small, actionable tasks.

The checklist should:

- Be organized by development phases (example: setup, data structures, functions, core logic, validation, testing).
- Contain small actionable tasks that I can complete and check off one by one.
- Include the relevant specification requirement beside each task.
- Include dependencies (what should be completed before another task).
- Mark critical requirements that are likely to cause grading issues.
- Include a final testing checklist to verify that the program fully complies with the specification.

Format example:



Phase 1: Project Setup

+ Ask anything



- Depends on: None

PDF

specifications

You are my programming assistant for a Python programming project. Your role is to help me strictly follow a given project specification file while guiding me through implementation, debugging, and improvement.

I have thoroughly read and analyzed the **PinoyBot** project specification. I am ready to act as your strict specification reviewer, programming assistant, and debugging guide. PDF

Below is my analysis of the project requirements, followed by an implementation checklist designed to keep us strictly on track.

1. Specification Analysis

Core Objective: Develop a word-level supervised machine learning classifier for Filipino code-switched text that identifies the language of each word. PDF

Input & Output Requirements:

- **Input:** A list of strings (tokens) representing a single Filipino passage. PDF
- **Output:** A list of predicted tags equal in length to the input list. PDF
- **Target Function:** `tag_language` inside `pinoybot.py`. PDF

Classification Tags:

+ Ask Gemini

Pro



Gemini is AI and can make mistakes.